

Computer Vision Session 1

Technical Document

1. Project Overview

This project implements two Computer Vision applications using Python, OpenCV, NumPy, and Object-Oriented Programming (OOP):

1. **Mini Painter**
2. **Object Counter**

The project was designed with a modular structure where each class has a specific responsibility. This improves code readability, maintainability, and extensibility.

2. Technologies and Libraries

2.1 Python

Python is used as the main programming language because it provides a simple syntax and has strong support for Computer Vision and numerical processing libraries.

2.2 OpenCV

OpenCV is the main Computer Vision library used in the project.

It is used for:

- Creating and displaying windows.
- Drawing shapes.
- Reading images.
- Converting color spaces.
- Applying thresholding.
- Detecting contours.
- Drawing bounding boxes.
- Handling mouse and keyboard events.

2.3 NumPy

NumPy is used for numerical and image-array operations.

It is used for:

- Creating the Mini Painter canvas.
- Representing image data.
- Creating HSV color boundaries.
- Defining polygon points.

3. Project Structure

The project is divided into two main applications.

CV_Session1/

```
|
|
|— miniPainter/
|  |— __init__.py
|  |— main.py
|  |— painter.py
|  |— canvas.py
|  |— color_manager.py
|  |— shapes/
|     |— shape.py
|     |— circle.py
|     |— rectangle.py
|     |— polygon.py
|
|— object_counter/
|  |— __init__.py
|  |— main.py
|  |— object_counter.py
|  |— image_processor.py
|  |— color_detector.py
|  |— contour_detector.py
|
|— test_images/
|  |— image.png
|
|— docs/
|  |— User Manual.docx
|  |— Technical Document.docx
|
```

├── README.md
├── requirements.txt
└── .gitignore

The two applications are kept separate because they have different responsibilities and processing pipelines.

4. Mini Painter

4.1 Architecture

Mini Painter follows a modular Object-Oriented design.

The main components are:

- Canvas
- ColorManager
- Painter
- Shape
- Circle
- Rectangle
- Polygon

The Painter class coordinates the application, while the other classes handle specific responsibilities.

4.2 Canvas

File

miniPainter/canvas.py

Responsibility

The Canvas class is responsible for:

- Creating the drawing canvas.
- Displaying the canvas.
- Saving the canvas.

The canvas is represented as a NumPy array with three color channels.

4.3 ColorManager

File

miniPainter/color_manager.py

Responsibility

The ColorManager class manages the available drawing colors.

The predefined colors are:

- White
- Red
- Green
- Blue
- Yellow

The class stores both the color value and its name.

4.4 Shape Abstraction

File

miniPainter/shapes/shape.py

The Shape class defines the common draw() behavior expected from all shapes.

The concrete implementations are:

Shape

└─ Circle

└─ Rectangle

└─ Polygon

Each shape implements its own drawing logic.

Circle

Uses cv.circle() to draw a filled circle.

Rectangle

Uses cv.rectangle() to draw a filled rectangle.

Polygon

Uses cv.fillPoly() to draw a filled triangular polygon.

4.5 Painter

File

miniPainter/painter.py

The Painter class is responsible for coordinating user interaction.

It handles:

- Current shape selection.
- Current color selection.
- Mouse input.
- Keyboard input.
- Status display.
- Saving and exiting.

The selected shape is stored as a Shape object and its draw() method is called when the user clicks the canvas.

5. Object Counter

5.1 Architecture

The Object Counter is divided into four main classes:

ObjectCounter

|

├── ImageProcessor

├── ColorDetector

└── ContourDetector

ObjectCounter acts as the main coordinator, while the other classes perform specific processing tasks.

6. Image Processing Pipeline

The Object Counter follows this processing pipeline:

Input BGR Image

↓

BGR → HSV

↓

Select Target Color

↓

HSV Thresholding

↓

Binary Mask

↓

Contour Detection



Contour Filtering



Bounding Boxes



Object Count

7. ImageProcessor

File

object_counter/image_processor.py

The ImageProcessor class handles basic image-processing operations.

It provides methods for:

Loading the image

load_image() uses OpenCV to load the image in BGR format.

Converting to HSV

convert_to_hsv() converts the BGR image to HSV using:

```
cv.cvtColor(image, cv.COLOR_BGR2HSV)
```

Creating a mask

create_mask() uses:

```
cv.inRange()
```

to create a binary mask based on the selected HSV range.

8. Why HSV Was Used

The project uses HSV instead of directly thresholding the BGR image.

HSV separates color information from brightness more clearly:

- **H (Hue):** represents the color.
- **S (Saturation):** represents the intensity or purity of the color.
- **V (Value):** represents brightness.

OpenCV represents HSV using:

H: 0–179

S: 0–255

V: 0–255

This makes HSV suitable for selecting objects based on their colors.

The current implementation supports:

- Red
- Green
- Blue

Each color has a predefined lower and upper HSV bound.

9. ColorDetector

File

object_counter/color_detector.py

The ColorDetector class provides the HSV boundaries for the selected color.

For example, when the user selects:

1 → Red

the class returns the predefined lower and upper HSV boundaries for red.

This keeps color-specific values separate from the main object-counting logic.

10. ContourDetector

File

object_counter/contour_detector.py

The ContourDetector class is responsible for detecting and processing contours.

10.1 Contour Detection

Contours are detected using:

`cv.findContours()`

The implementation uses:

`cv.RETR_EXTERNAL`

to retrieve only external contours.

This is suitable for counting separate colored objects without processing internal contours.

10.2 Contour Filtering

Small contours can be caused by noise or very small regions.

A minimum contour area is therefore used:

`MIN_CONTOUR_AREA = 500`

Only contours whose area is greater than or equal to this value are counted.

10.3 Bounding Boxes

For every valid contour, the application uses:

```
cv.boundingRect()
```

to determine the object's bounding rectangle.

The rectangle is then drawn around the detected object.

11. ObjectCounter

File

object_counter/object_counter.py

The ObjectCounter class coordinates the complete detection process.

It receives the required components through its constructor:

```
ObjectCounter(  
    image_processor,  
    color_detector,  
    contour_detector  
)
```

This allows the main class to use the required services without creating them internally.

The main responsibilities are:

- Loading the image.
 - Converting it to HSV.
 - Selecting the target color.
 - Creating the mask.
 - Detecting contours.
 - Counting valid objects.
 - Displaying the results.
-

12. Object-Oriented Programming

The project follows Object-Oriented Programming principles by separating functionality into classes.

Examples include:

- Canvas handles canvas operations.

- ColorManager handles color selection.
- Painter handles application interaction.
- ImageProcessor handles image processing.
- ColorDetector handles color boundaries.
- ContourDetector handles contour detection.
- ObjectCounter coordinates the object-counting workflow.

This separation keeps each class focused on a specific responsibility.

13. SOLID Principles

The project applies SOLID principles where appropriate.

13.1 Single Responsibility Principle

Each class has a focused responsibility.

For example:

- Canvas → canvas operations.
- ColorManager → color management.
- ImageProcessor → image processing.
- ContourDetector → contour detection.

This prevents one class from becoming responsible for unrelated operations.

13.2 Open/Closed Principle

The design allows new functionality to be added with minimal modification to existing code.

For example, a new shape can be added by creating another class derived from Shape without changing the existing Circle, Rectangle, or Polygon classes.

Similarly, new color ranges can be added to the color-detection component without changing the complete object-counting pipeline.

13.3 Liskov Substitution Principle

The concrete shape classes:

Circle

Rectangle

Polygon

can be used through the Shape abstraction.

The Painter does not need to know the internal implementation of each shape. It only needs to call:

```
draw()
```

13.4 Interface Segregation Principle

The project does not introduce large or unnecessary interfaces.

The abstractions used are kept small and focused on the required functionality.

This is appropriate for the size and scope of the project.

13.5 Dependency Inversion Principle

The ObjectCounter receives its dependencies through its constructor:

```
ObjectCounter(  
    image_processor,  
    color_detector,  
    contour_detector  
)
```

This is a simple form of Dependency Injection.

The high-level object-counting workflow does not need to create the processing components internally.

This makes the design easier to modify and test.

14. Design Decisions

14.1 Separate Classes

The project was divided into multiple classes instead of putting all functionality in one file.

This was done to:

- Improve readability.
- Follow SRP.
- Make the code easier to maintain.
- Make future extensions easier.

14.2 HSV Instead of Grayscale

Grayscale thresholding was not used for color-specific detection because converting an image to grayscale removes color information.

HSV was selected because the assignment requires color-based detection and HSV provides a more suitable representation for selecting colors.

14.3 External Contours

`cv.RETR_EXTERNAL` was selected because the application is interested in counting separate external objects rather than internal contour structures.

14.4 Minimum Contour Area

A minimum contour area was introduced to reduce the effect of small unwanted regions and noise.

The current threshold is:

`MIN_CONTOUR_AREA = 500`

This value can be adjusted depending on the input image.

14.5 Dependency Injection

The main Object Counter class receives its processing components through its constructor.

This keeps object creation separate from the main counting workflow and supports a cleaner modular design.

15. Modifications and Improvements

During development, several implementation decisions and fixes were made.

Mini Painter

- Added a proper application entry point in `main.py`.
- Added shape selection using keyboard input.
- Added color selection using predefined colors.
- Added mouse-based drawing.
- Added status information showing the current shape and color.
- Added saving functionality.
- Organized shapes using a common Shape abstraction.
- Added package initialization files where required.

Object Counter

- Added BGR image loading.
- Added BGR-to-HSV conversion.
- Added predefined HSV ranges for red, green, and blue.
- Added binary mask generation.
- Added contour detection.

- Added contour-area filtering.
 - Added bounding boxes.
 - Added object counting.
 - Added keyboard-based color selection.
 - Added support for switching between colors without restarting the program.
 - Added a test image for validation.
-

16. Conclusion

The project demonstrates the implementation of basic Computer Vision tasks using Python, OpenCV, and NumPy while applying Object-Oriented Programming principles.

The Mini Painter demonstrates interactive image drawing and event handling, while the Object Counter demonstrates color-based image segmentation, contour detection, and object counting.

The modular structure and use of SOLID principles provide a maintainable foundation that can be extended with additional shapes, colors, or image-processing techniques.